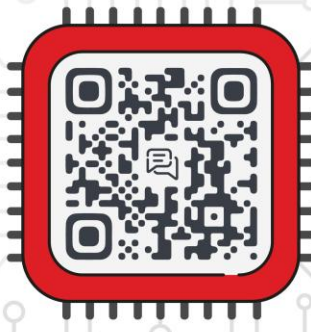




نظري

3



14



1680 SP

كلية الهندسة المعلوماتية

السنة الثالثة

Pipelining

د. سيرا أستور



RB Informatics; 27/10/2025

بنيان الحواسيب 2

في هذه المحاضرة سنتحدث عن أهم التقنيات المستخدمة في تحسين أداء المعالج و التي تدعى ال Pipelining

تطور المعالجات:

■ في البداية كان لدينا المعالج أحادي الدورة (Single-Cycle) :

ينفذ كل تعليمة كاملة في دورة ساعة واحدة

بسيط في البنية لكنه بطيء لأن كل تعليمة تأخذ وقت طويل

■ ثم ظهر المعالج متعدد الدورات (Multi-Cycle):

يقسم تنفيذ التعليمة إلى عدة مراحل، كل مرحلة في دورة مستقلة

يقلل زمن الدورة، لكنه ما زال ينفذ تعليمة واحدة فقط في نفس الوقت

■ بعد ذلك انتقلنا إلى خط الأنابيب (Pipelining) وهو ما ستناوله في محاضرتنا

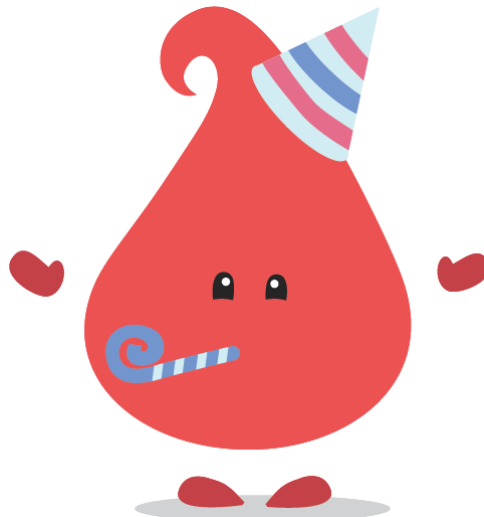
يسمح بتنفيذ عدة تعليمات بشكل متداخل (كل تعليمة في مرحلة مختلفة).

الهدف الأساسي هو زيادة معدل الإنجاز (Throughput)

■ وأخيراً وصلنا إلى التوازي (Parallelism) :

تنفيذ أكثر من تعليمة أو أكثر من خيط (Thread) في نفس اللحظة باستخدام وحدات متعددة

يرفع الأداء بشكل كبير من خلال استغلال تعدد الموارد



النوع	عدد التعليمات في نفس الوقت	زمن الدورة	التعقيد البنيوي	الأداء
أحادي الدورة	تعليمية واحدة فقط	طويل	بسيط	منخفض
متعدد الدورات	تعليمية واحدة فقط	قصير	متوسط	أفضل قليلا
Pipelining	عدة تعليمات متداخلة	قصير	أعقد	أعلى
Parallelism	عدة تعليمات متوازية	قصير	عالي جداً	الأعلى

في المحاضرات السابقة درسنا البنية الأحادية ومتعددة الدورة في مادة بنیان الحواسيب 1- أما في هذه المحاضرة والمحاضرات القادمة فسنناول بالتفصيل الموضوعي Parallelism و Pipelining

نلاحظ أن Parallelism من حيث الأداء هو الأعلى لكنه عالي التكلفة، لذلك نقوم باستخدام ال Pipelining لأنه يوفر لنا حل اقتصادي هندسي

الحل الهندسي هو : الحل الذي يحل المشكلة بأقل تكاليف ممكنة

التوازي على مستوى التعليمات ILP : Instruction Level Parallelism

في المعالجات ذات الدورة الواحدة أو المتعددة، يتم تنفيذ تعليمية واحدة فقط في كل دورة.

• لذا هنا كيف يمكننا الحصول على أداء أفضل؟

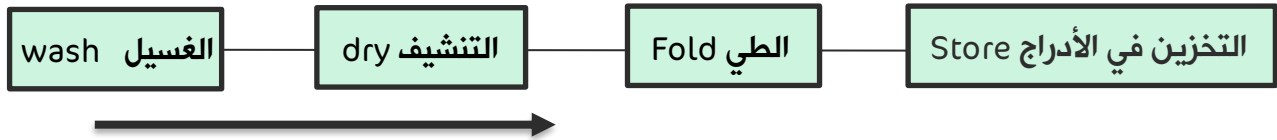
الجواب : بتنفيذ عدة تعليمات في نفس الوقت ، و الطرق الأساسية لذلك :

- Pipelining (خط الأنابيب) : تحسين المعالج متعدد الدورات بحيث يجلب تعليمية جديدة في كل دورة.
- Parallelism (التوازي) : جلب عدة تعليمات في نفس الدورة.
- الفكرة الأساسية : التداخل بين التعليمات أثناء التنفيذ يُعرف باسم Instruction-Level Parallelism (ILP)



لتوضيح الفكرة سنأخذ مثال الغسيل (Laundry Example)

ليكن لدينا أربع دفعات غسيل A , B , C , D
كل دفعة تمر بأربع عمليات أساسية :



زمن كل عملية = 30 دقيقة

إذا عمل شخص واحد على كل دفعة كاملة، سيستغرق إنجاز دفعة واحدة 120 دقيقة.

نسمي عدد التعليمات في الدفعة الواحدة بـ n .

زمن اكمال دورة واحدة = n وحدة زمنية.

نسمي عدد التعليمات المنفذة في واحدة الزمن بمعدل الإنجاز (Throughput).

معدل الإنجاز = $\frac{1}{n}$ وحدة زمنية

في مثالنا : معدل الإنجاز = $\frac{1}{4}$ وحدة زمنية (30 دقيقة).

أي خلال 30 دقيقة يتم إنجاز ربع دفعة.

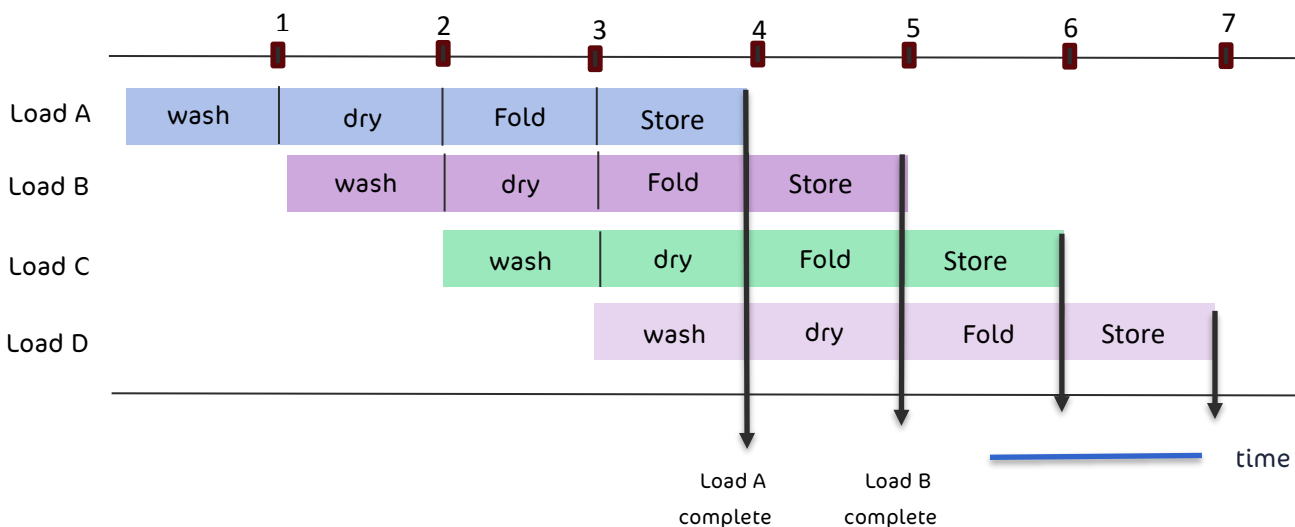
عند توزيع العمل على شكل خط إنتاج Assembly Line :

(1) أثناء غسيل الدفعة الأولى يمكن تنشيف الثانية

(2) في نفس الوقت يمكن طي الثالثة

(3) وأيضاً تخزين الرابعة

بهذه الطريقة تصبح العمليات متداخلة، ويزداد معدل الإنجاز (Throughput).



من المخطط نجد ان :

- نلاحظ ان يوجد زمن ضائع في تعبئة وتفريغ ال pipeline نسويه cycle drain وهو زمن ثابت مقداره $n-1$.
- زمن انهاء الدفعة الأولى = n وحدة زمنية.
- عدد الدفعات المكتملة في اللحظة $T-n+1 = T$
- فيكون معدل الإنجاز:
- $\text{Throughput} = 1 - \frac{n-1}{T}$ Loads per unit time
- $\frac{\text{Throughput(assembly line)}}{\text{Throughput(team assembly)}} = \frac{1 - \frac{n-1}{T}}{\frac{1}{n}} = n - \frac{n(n-1)}{T} \rightarrow n \text{ as } T \rightarrow \infty$
- أي كلما زاد عدد الدفعات المطلوبة بالتالي يزداد الزمن اللازم يكون مقدار التحسين يقترب من n .

النتيجة: زمن إنجاز أول دفعة يبقى 120 دقيقة، لكن بعد امتلاء خط الإنتاج نحصل على دفعة جديدة كل 30 دقيقة.
هذا يشبه تماماً فكرة Pipelining في المعالجات حيث يتم تحسين معدل الإنجاز عبر التداخل بين المراحل.

■ عند استخدام خط الإنتاج (Assembly Line) :

- بعد امتلاء الخط، نحصل على دفعة جديدة كل 30 دقيقة
- أي أن معدل الإنجاز يصبح دفعة واحدة كل 30 دقيقة

الخلاصة :

زمن إكمال أول دفعة يبقى كما هو (120 دقيقة).

لكن معدل الإنجاز الكلي يتحسن بشكل كبير بفضل التداخل بين العمليات.

هذا يوضح كيف يعمل ال Pipelining في المعالجات: تحسين معدل الإنجاز دون تقليل زمن تنفيذ التعليمة الواحدة.



خصائص المعالج بخط الأنابيب

1. جميع الوحدات الوظيفية تعمل بشكل مستقل.
 2. يسمح بتنفيذ عدة مهام بشكل متزامن باستخدام موارد مختلفة.
 3. خط الأنابيب لا يقلل زمن تنفيذ التعليمات الواحدة (Latency)، لكنه يحسن معدل الإنجاز الكلي (Throughput).
 4. التسريع المحتمل = عدد مراحل خط الأنابيب.
 5. معدل التنفيذ في الخط يعتمد على :
 - أبطأ مرحلة في الخط.
 - زمن ملء الخط وزمن تفريغه.
 - عدم توازن طول المراحل (إذا كانت مرحلة أطول من الأخرى، المراحل الأخرى تنتظر).
 6. قد يحدث توقف (Stall) بسبب الارتباط بين التعليمات (Dependencies) .
- مثال : $A = B + C$
 $L = A + M$
- التعليمات الثانية لا تستطيع أن تنفذ إلا بعد انتهاء التعليمات الأولى بالكامل .

(Pipelining in a Computer)

- يتم تقسيم المسار (Datapath) إلى مهام متقاربة في الزمن، تُنفَّذ بشكل تسلسلي باستخدام موارد غير متداخلة.
- يتم إدخال سجلات (Registers) عند حدود المهام داخل المسار؛ هذه السجلات تنقل ناتج كل مهمة ليكون مدخل المهمة التالية.
- يتم مزامنة المهام باستخدام ساعة (Clock) بزمن دوره يساوي تقريباً أطول مهمة.
- كل تعليمات تُجرأ إلى مجموعة مهام صغيرة، بحيث يمكن تنفيذ التعليمات بشكل متداخل (Staggered) .

النتيجة: المعالج يستطيع تنفيذ عدة تعليمات في نفس الوقت، كل واحدة في مرحلة مختلفة من خط الأنابيب.

مميزات تصميم تعليمات MIPS بالنسبة لـ Pipeline

1. طول التعليمات ثابت

- جميع تعليمات MIPS لها نفس الطول (32 بت).
- هذا يجعل عملية جلب التعليمات (Instruction Fetch) في المرحلة الأولى أسهل، وكذلك فك الترميز (Decode) في المرحلة الثانية.

2. عدد قليل من صيغ التعليمات (Instruction Formats)

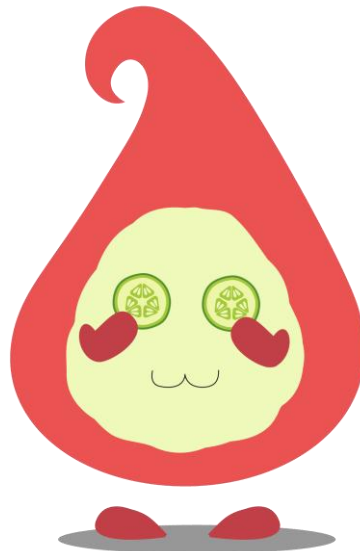
- الحقول الخاصة بالسجلات (Source Registers) موجودة دائماً في نفس المكان داخل التعليمات.
- هذه التماثلية (Symmetry) تسمح للمرحلة الثانية أن تبدأ قراءة سجلات المعالج في نفس الوقت الذي يتم فيه تحديد نوع التعليمات.
- لو لم تكن الصيغ متماثلة، لكان علينا تقسيم المرحلة الثانية إلى مرحلتين إضافيتين، مما يؤدي إلى ست مراحل بدلاً من خمس.

3. المعاملات المرتبطة بالذاكرة تظهر فقط في تعليمات التحميل والتخزين (Load/Store)

- هذا التقييد يجعلنا نستخدم مرحلة التنفيذ (EX) لحساب عنوان الذاكرة، ثم نصل إلى الذاكرة في المرحلة التالية (MEM).
- النتيجة: لا حاجة لتكرار الوصول إلى الذاكرة في أكثر من مرحلة.

4. المعاملات يجب أن تكون مصطفة (Aligned) في الذاكرة

- هذا يعني أن عملية نقل البيانات بين المعالج والذاكرة يمكن أن تتم في مرحلة واحدة فقط.
- لا يوجد خطر أن تحتاج تعليمة واحدة إلى عمليتي وصول للذاكرة في نفس الوقت.



Pipelining a Single-Cycle Datapath

الجدول التالي مثال يوضح العديد من التعليمات و مراحل تنفيذها في ال single cycle :

Instruction class	Instr. fetch (IF)	Instr. Decode (also reg. file read) (ID)	Execution (ALU Operation) (EX)	Data access (MEM)	Write Back (Reg. file write) (WB)	Total time
lw	2ns	1ns	2ns	2ns	1ns	8ns
sw	2ns	1ns	2ns	2ns		8ns
R-format add, sub, and, or, slt	2ns	1ns	2ns		1ns	8ns
B-format, beq	2ns	1ns	2ns			8ns

ماذا تلاحظ ؟:

- نلاحظ أن أطول تعليمة هي تعليمة ال Load لأنها تمر بكل الأقسام
- تعليمة Store تخزن في الذاكرة فقط ولا تعيد التحميل لذلك آخر مرحلة لا تمر بها .
- نلاحظ أن تعليمة R-type لا تمر بالذاكرة
- تعليمات التفرع و القفز هي أقصر التعليمات .

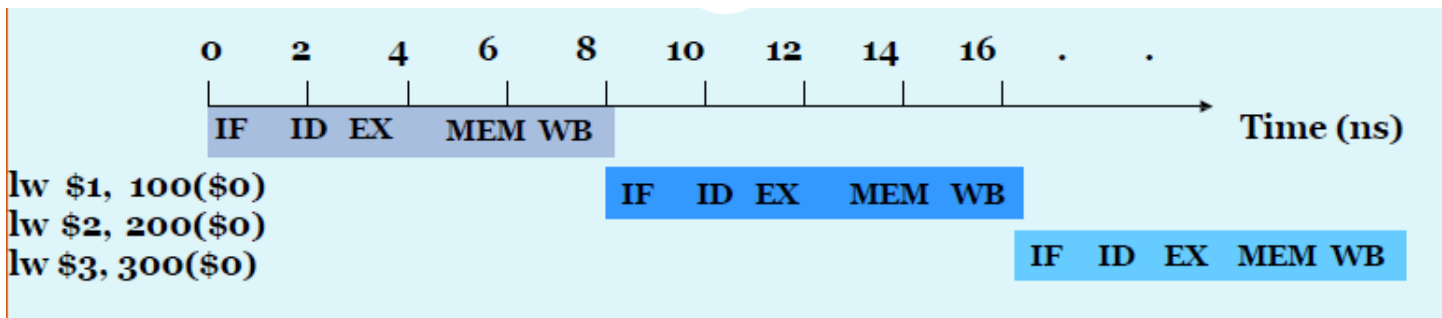
الملاحظات المهمة:

كل التعليمات تقريباً تستغرق نفس الزمن الكلي (8ns)، حتى لو كان هناك بعض المراحل التي لا تمر بها (اللون الأصفر)

الهدف من هذا التنظيم هو جعل كل التعليمات تمر بخمس مراحل متساوية، مما يسهل عملية ال Pipelining و يزيد الأداء .



Execution Time :Single-Cycle



فرضا لدينا ثلاث تعليمات (lw) load متتالية :

❖ زمن كل منها هو 8 ns Clock cycle time =

❖ زمن التعليمات الثلاث إذا تم تنفيذها بشكل متسلسل هو 24ns

هذا الزمن سيكون كبير نسبياً إذا ما كان عدد التعليمات أكثر .

Pipelined Datapath " Single Cycle"

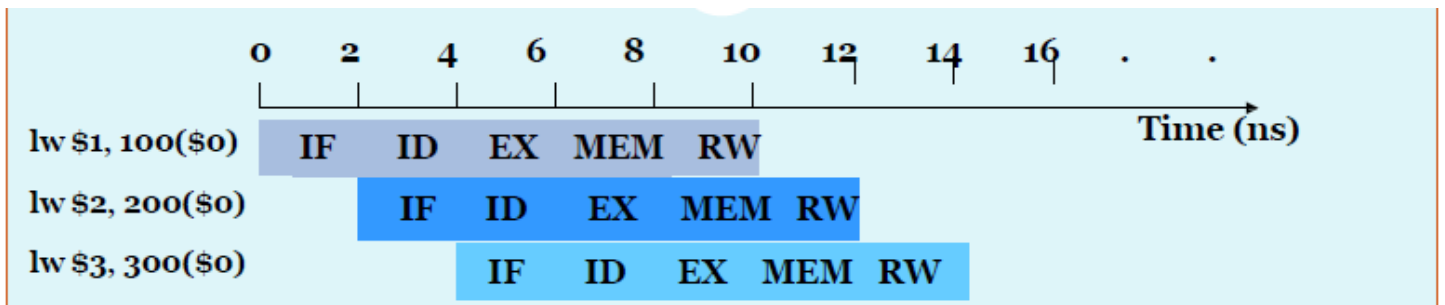
↩ Single cycle بعد التوارد pipelining :

Instruction class	Instr. fetch (IF)	Instr. Decode (also reg. file read) (ID)	Execution (ALU Operation) (EX)	Data access (MEM)	Write Back (Reg. file write) (WB)	Total time
lw	2ns	1ns 2ns	2ns	2ns	1ns 2ns	10ns
sw	2ns	1ns 2ns	2ns	2ns	1ns 2ns	10ns
R-format: add, sub, and, or, slt	2ns	1ns 2ns	2ns	2ns	1ns 2ns	10ns
B-format: beq	2ns	1ns 2ns	2ns	2ns	1ns 2ns	10ns

نلاحظ أننا جعلنا جميع المراحل تأخذ نفس التوقيت فكل شيء يصبح 2ns

- نلاحظ أن الزمن الكلي للتعليمية ازداد total time = 10 ns (لكن التحسين يتمثل في الإنتاجية الكلية)
- الأماكن الفارغة أصبحت ممتلئة و ذلك بسبب التوارد .

Execution Time : Pipelined Single Cycle



زمن تنفيذ التعليمة ازداد بمقدار 2 ns لكن الوقت اللازم لتنفيذ تعليمات ال load الثلاث هو 14ns

بالمقارنة بين الجدولين السابقين نلاحظ أنه عندما جعلنا جميع مراحل التعليمة تأخذ نفس الوقت، حصلنا على وقت أقل في تنفيذ نفس العدد من التعليمات

نسبة التحسين :

$$performance = \frac{single\ cycle\ time}{pipeline\ time} = \frac{24}{14} = 1.7$$

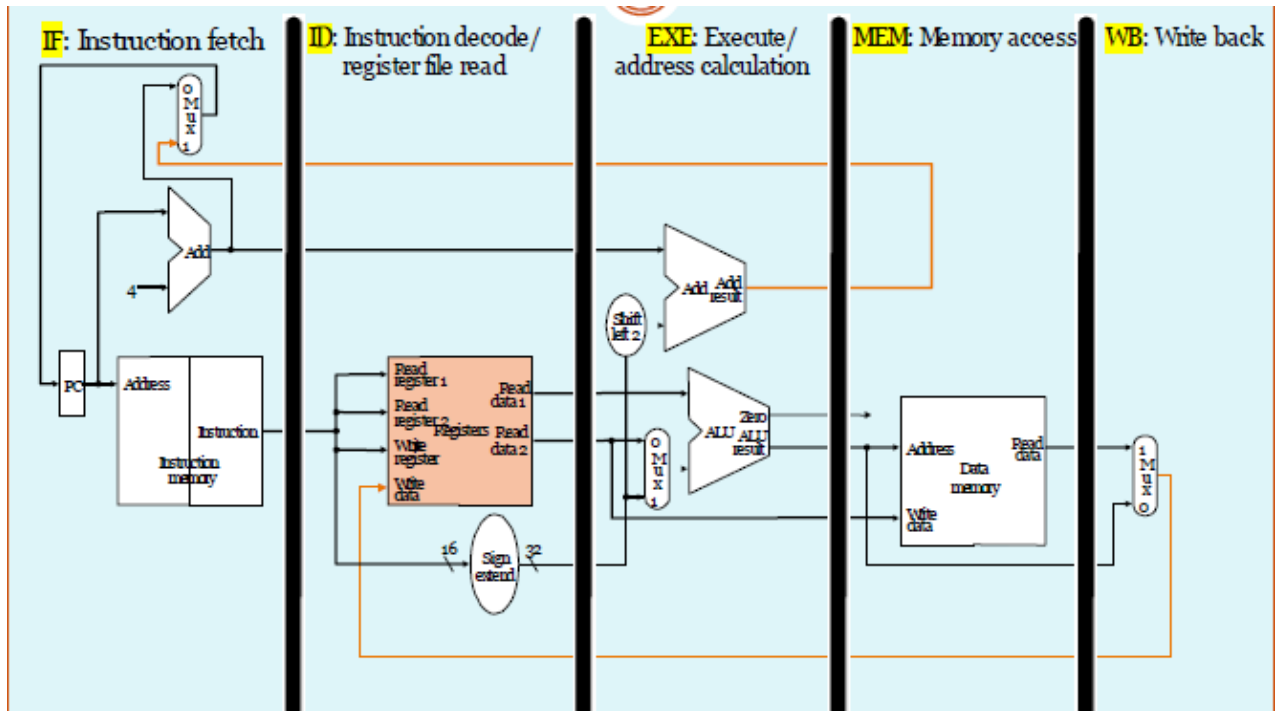
يعطى قانون ال speed up في ال pipeline كالتالي

$$time\ between\ instructions\ pipelined = \frac{time\ between\ instruction\ nonpipelined}{Number\ of\ pipe\ stages}$$

القانون وظيفته هو حساب الربح بال Pipeline مقارنة بال non Pipelined



مراحل التعليم



الخطوط السوداء تمثل الذواكر التي يجب أن أحفظ بها التعليمات مابين المرحلة والمرحلة الأخرى لدينا خمس مراحل فسنحتاج إلى أربع ذواكر

المراحل الأساسية للتنفيذ (Pipeline Stages)

IF – Instruction Fetch

جلب التعليم من الذاكرة..

ID – Instruction Decode & Register Read

فك ترميز التعليم وقراءة السجلات

EX – Execution (ALU Operation)

تنفيذ العملية الحسابية أو حساب العنوان

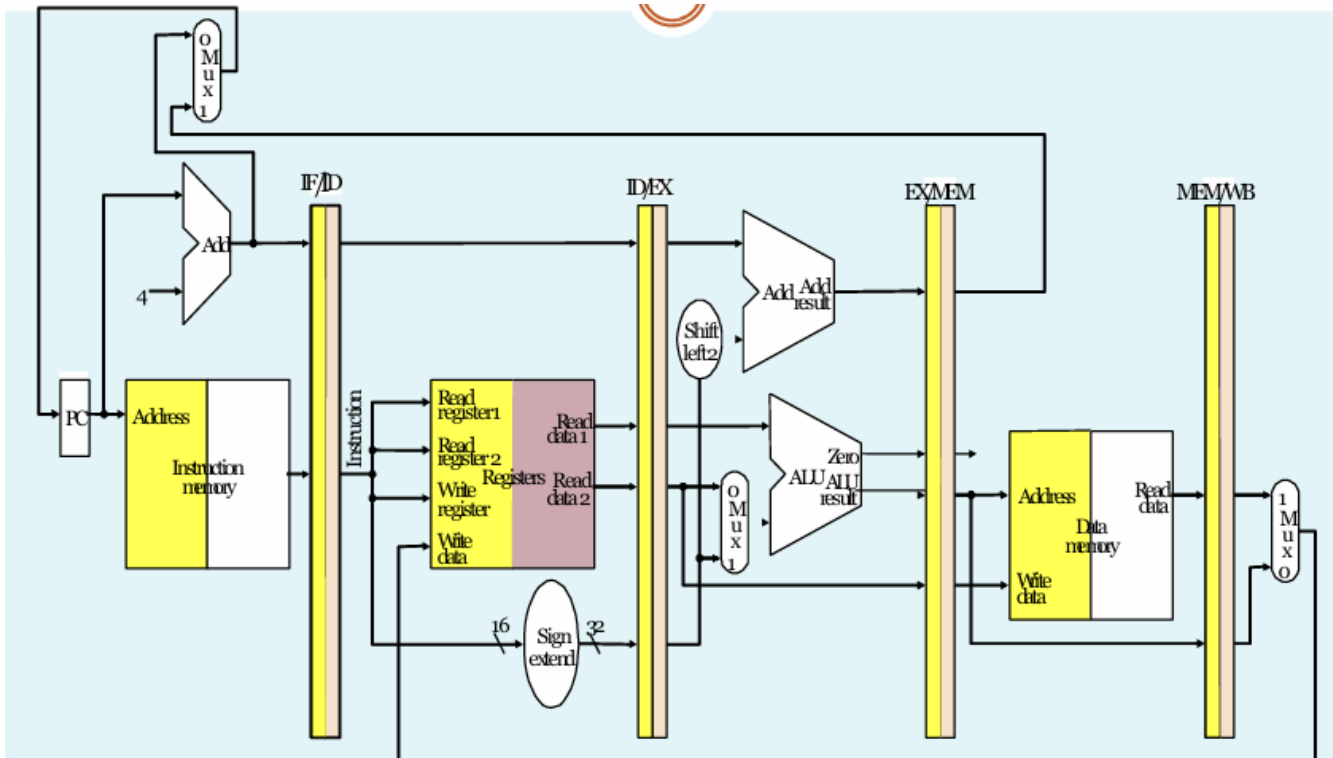
MEM – Memory Access

الوصول إلى الذاكرة (قراءة أو كتابة).

WB – Write Back

كتابة النتيجة في سجل المعالج

Pipelined Single Cycle Registers Adding



نسمي المسجلات حسب موقعها من تنفيذ التعليمة كما قسمنا المسجل إلى قسمين، القسم الأول (الأصفر) هو خرج (output) للمرحلة الأولى والقسم الثاني هو دخل (input) للمرحلة الثانية

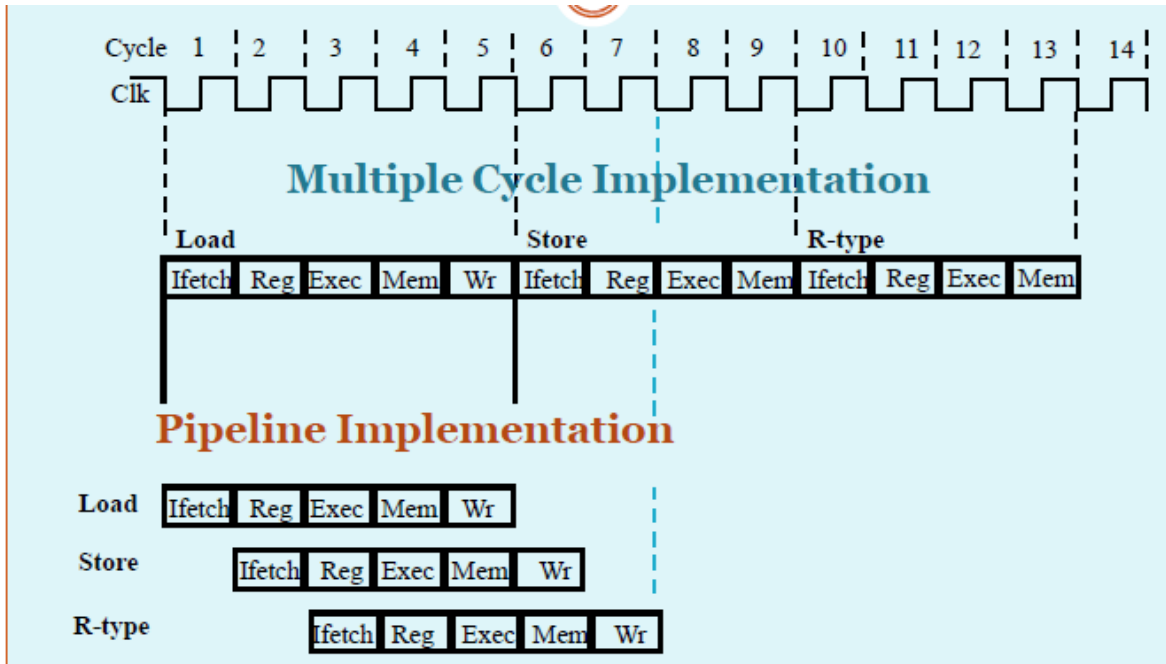
تمت تسميتهم بحسب المراحل فمثلا IF/ID أي هو المسجل الموجود بين مرحلة الجلب و مرحلة فك الترميز

ولكي نعرف كل مسجل الحجم الذي يحتاجه نقوم بفهم ماذا يخزن

الجدول التالي يوضح ما تخزنه السجلات في كل مرحلة :

Register Name	Data Held
IF/ID	PC+4, Instruction Word (IW)
ID/EX	PC+4, R1, R2, IW(0-15) sign ext., IW(11-15), IW(16-20)
EX/MEM	PC+4, zero, ALUResult, R2, IW(11-15) OR IW(16-20)
MEM/WB	M[ALUResult], ALUResult, IW(11-15) OR IW(16-20)

Multiple Cycle VS. Pipeline



في ال Multi Cycle
فقط قسمنا النبضة
لخمسة نبضات و
نلاحظ أنه بقي الزمن
طويل

أما بعد تطبيق
التوارد :

حساب زمن التنفيذ في أنواع مختلفة من المعالجات:

زمن الدورة في أحادي المعالج الدورة (Single Cycle) هو 45 ns نانو ثانية.
زمن الدورة في المعالج متعدد الدورات (Multi-Cycle) والمعالج المتوازي (Pipelined) هو 10 ns نانو ثانية.
متوسط عدد الدورات لكل تعليمة (CPI) في المعالج متعدد الدورات نتيجة تنوع التعليمات هو 4.6.
المطلوب:

حساب زمن التنفيذ لكل نوع من المعالجات عند تنفيذ 100 تعليمة.

1. المعالج أحادي الدورة (Single Cycle Machine):

كل تعليمة تنفذ في دورة واحدة.

$$IC \times CPI \times \text{clock cycle time}$$

$$\text{الزمن} = 100 \times 1 \text{ CPI} \times 45 \text{ ns/cycle}$$

النتيجة: 4500 ns

2. المعالج متعدد الدورات (Multi-Cycle Machine):

كل تعليمة تحتاج في المتوسط إلى 4.6 دورة.

$$\text{الزمن} = 100 \times 4.6 \times 10 \text{ ns/cycle}$$

النتيجة: 4600 ns

ليس بالضرورة ان يكون ال Single Cycle Machine أسرع من ال Multi-Cycle Machine وذلك يعتمد على مزيج التعليمات (instruction mix)

3. المعالج المتوازي (Pipelined Machine):

كل تعليمة تنفذ في دورة واحدة بعد ملء خط الأنابيب.

هناك تأخير إضافي بمقدار 4 دورات لتفريغ خط الأنابيب (Drain Cycles).

$$\text{الزمن} = (100 \times 1 \text{ CPI} + 4 \text{ دورات تفريغ}) \times 10 \text{ ns/cycle}$$

النتيجة: 1040 ns

فيكون التحسين:

$$\text{performance} = \frac{\text{single cycle time}}{\text{pipeline time}} = \frac{4500}{1040} = 4.32$$

أي تحسن الأداء بنسبة 332٪

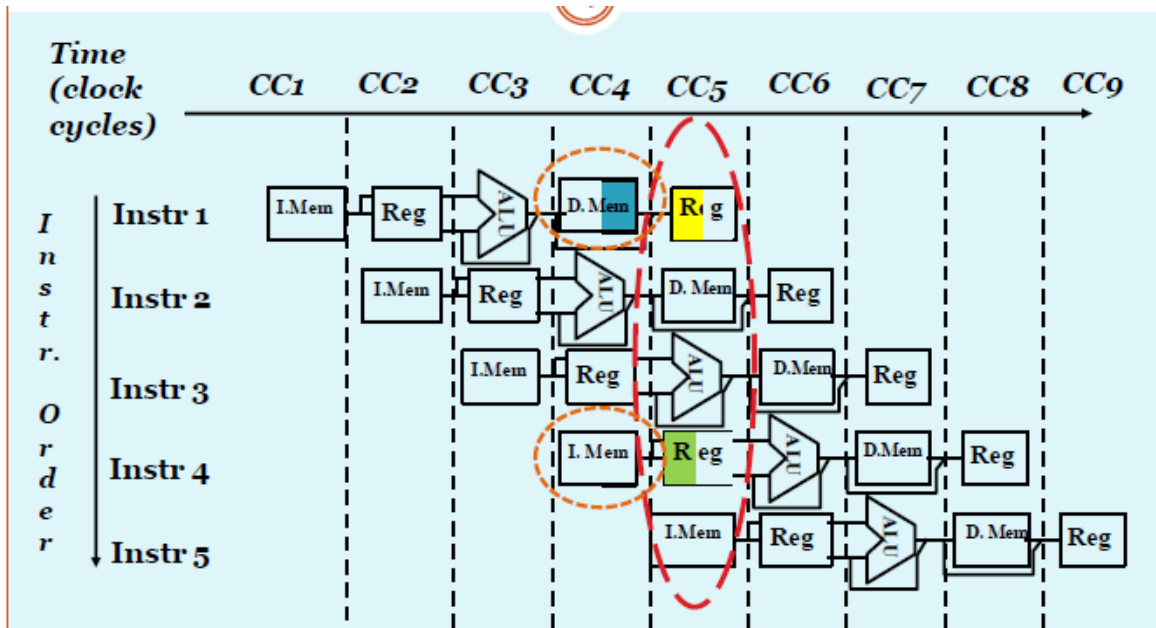
دورات التفريغ (cycle drain) :

وهي تساوي $n-1$ حيث n عدد مراحل المقسمة لها التعليمة (Fetch, Decode, Execute, Memory, Write)

$$\text{Cycle_Drain} = n - 1 = 5 - 1 = 4 \quad \text{ف} \quad n = 5$$



Graphical Representation- SC (single cycle)



نلاحظ أنه يحدث في جزء من المعالج تنفيذ عمليتي القراءة والكتابة على نفس الذاكرة أو المسجل في الوقت ذاته وهذا الشيء يسبب لي مشاكل نسميها Pipeline Hazards

Pipeline Hazards

مشاكل ال Pipeline هي حالات تؤدي إلى عدم قدرة المعالج على تنفيذ التعليمات التالية مباشرة بعد انتهاء التعليمات الحالية في دورة الساعة التالية. هذه المشكلات تؤثر على الأداء وتسبب تأخيرًا في التنفيذ.

أنواع المشكلات

1. المخاطر الهيكلية (Structural Hazard):

مثال: إذا كانت هناك تعليمتان تحتاجان إلى استخدام وحدة الضرب في نفس الوقت، فإن واحدة منهما ستنتظر حتى تنتهي الأخرى.

2. مخاطر البيانات (Data Hazard):

تنشأ عندما تعتمد تعليمة على نتيجة تعليمة سابقة لم تنفذ بعد.

مثال: إذا كانت التعليمة الثانية تحتاج إلى قيمة ناتجة من التعليمة الأولى، ولكن هذه القيمة لم تحسب بعد.

3. مخاطر التحكم (Control Hazard):

مثال: عند تنفيذ تعليمة "if" أو "branch"، لا يعرف المعالج أي تعليمة ينفذ بعدها حتى يتم تحديد نتيجة الشرط، مما يسبب تأخيرًا.



The end